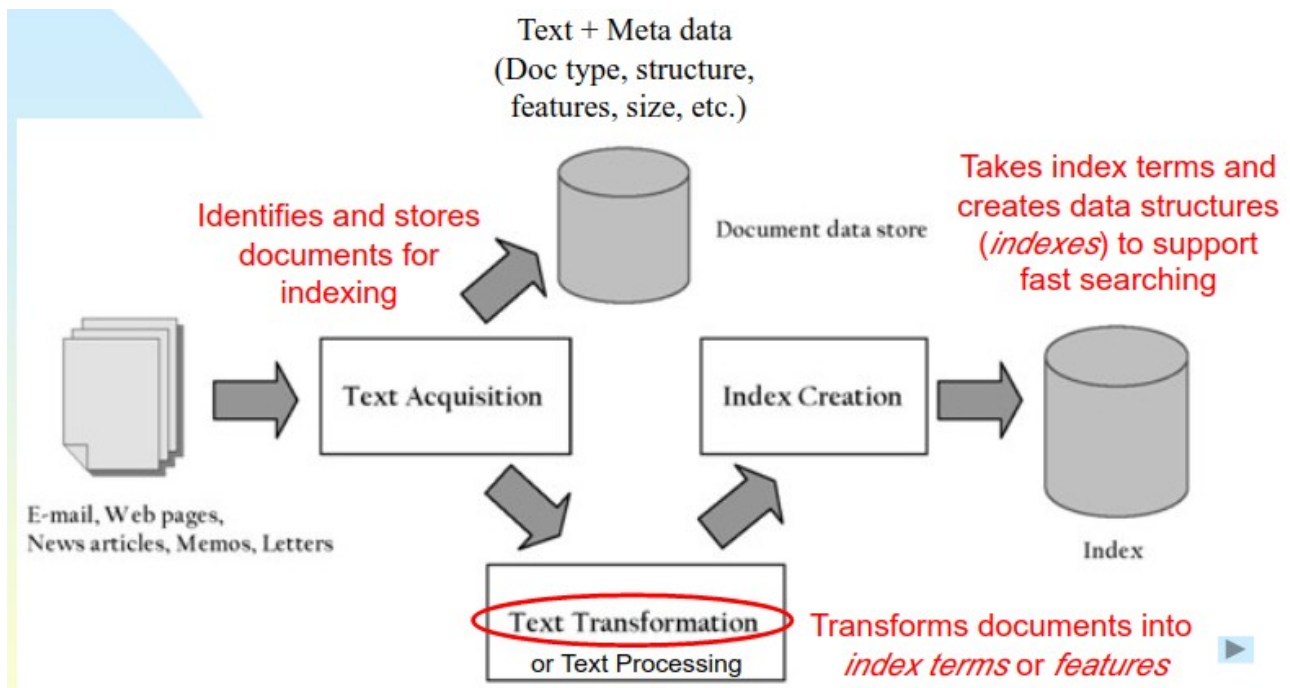


2 Processing Text (Text Transformation)



2.1 From Words to Terms

- Modifying/Converting the many forms in which words can occur into more consistent index terms that represent the content of a document
- What are the problems?
 - Matching the exact string of characters typed by the user is too restrictive
 - Examples
 - Case sensitivity: How to handle words with different cases (upper/lower case)
 - How to handle punctuations
 - Stemming: How to handle words with the same stem
 - Sometimes not clear where words begin and end
 - Not even clear what a word is in some languages, e.g., in Chinese and Korean
 - Not all words are of equal value in a search, and understanding the statistical nature of text is critical

2.2 Text Statistics

- Huge variety of words used in text but many statistical characteristics of word occurrences are predictable
 - e.g., distribution of word counts
- Retrieval models and ranking algorithms depend heavily on statistical properties of words
 - e.g., important/significant words occur often in documents but are not high frequency in collection

2.2.1 Zipf's Law

- Distribution of word frequencies is very skewed
 - Few words occur very often, many hardly ever occur
 - Examples
 - “the” and “of”, two common words, make up about 10% of all word occurrences in text documents
 - The most frequent six words account for 20% of occurrences
 - The most frequent 50 words are about 40% of all text
- Zipf's law
 - The frequency f of a word in a corpus is inversely proportional to its rank r (assuming words are ranked in order of decreasing frequency)

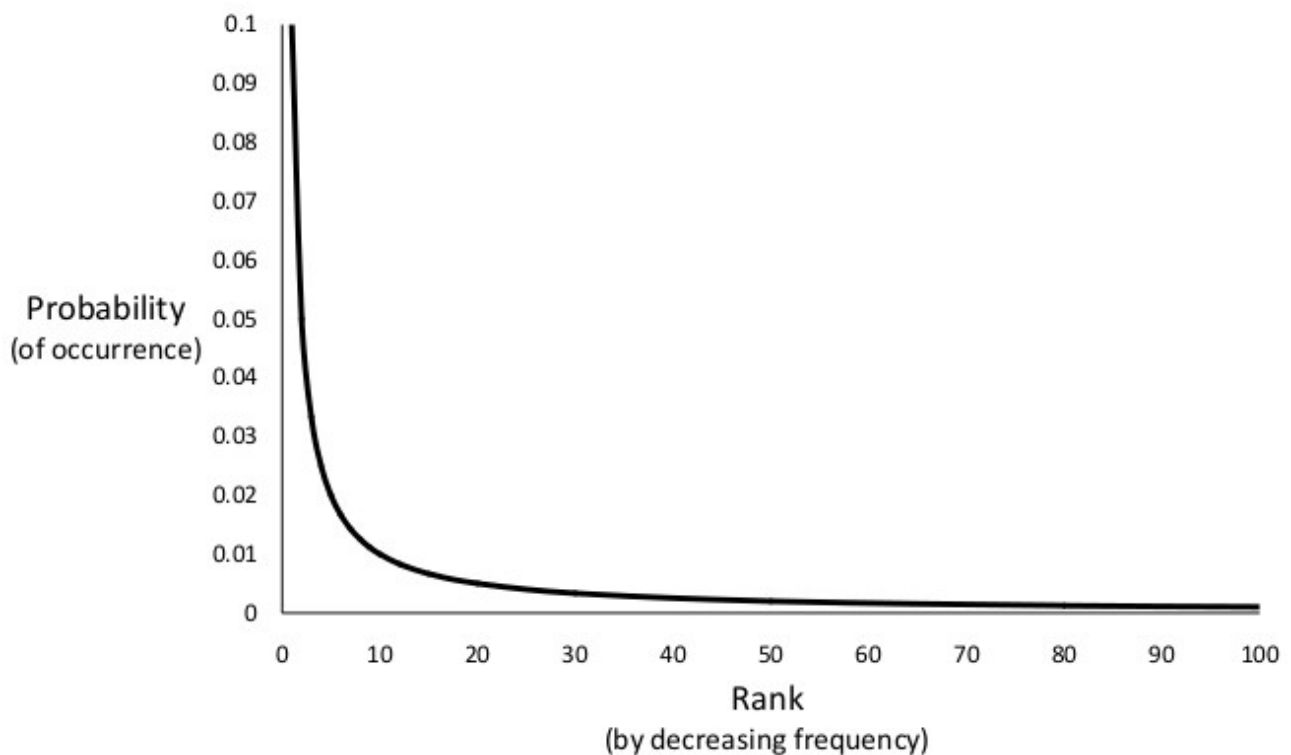
$$r \cdot f = k$$

where k is a constant for the corpus

- We often want to talk about the probability of occurrence of a word (which is just the frequency of the word divided by the total number of word occurrences in the text)
- Hence, can rewrite Zipf's law as

$$r \cdot P_r = c$$

- For English, $c \approx 0.1$



Example

Zipf's law prediction of word occurrences in the Associated Press collection of news stories from 1989 (AP89)

Statistics for the AP89 collection

Total documents	84,678
Total word occurrences	39,749,179
Vocabulary size	198,763
Words occurring > 1000 times	4,169
Words occurring once	70,064

- Vocabulary size: the number of unique words in the collection (nearly 200,000)
- A large proportion of these words (70,000) occur only once
 - Words that occur once in a text corpus or book have long been regarded as important in text analysis,

Most frequent 50 words for the AP89 collection

<i>Word</i>	<i>Freq.</i>	<i>r</i>	<i>P_r(%)</i>	<i>r.P_r</i>	<i>Word</i>	<i>Freq</i>	<i>r</i>	<i>P_r(%)</i>	<i>r.P_r</i>
the	2,420,778	1	6.49	0.065	has	136,007	26	0.37	0.095
of	1,045,733	2	2.80	0.056	are	130,322	27	0.35	0.094
to	968,882	3	2.60	0.078	not	127,493	28	0.34	0.096
a	892,429	4	2.39	0.096	who	116,364	29	0.31	0.090
and	865,644	5	2.32	0.120	they	111,024	30	0.30	0.089
in	847,825	6	2.27	0.140	its	111,021	31	0.30	0.092
said	504,593	7	1.35	0.095	had	103,943	32	0.28	0.089
for	363,865	8	0.98	0.078	will	102,949	33	0.28	0.091
that	347,072	9	0.93	0.084	would	99,503	34	0.27	0.091
was	293,027	10	0.79	0.079	about	92,983	35	0.25	0.087
on	291,947	11	0.78	0.086	i	92,005	36	0.25	0.089
he	250,919	12	0.67	0.081	been	88,786	37	0.24	0.088
is	245,843	13	0.65	0.086	this	87,286	38	0.23	0.089
with	223,846	14	0.60	0.084	their	84,638	39	0.23	0.089
at	210,064	15	0.56	0.085	new	83,449	40	0.22	0.090
by	209,586	16	0.56	0.090	or	81,796	41	0.22	0.090
it	195,621	17	0.52	0.089	which	80,385	42	0.22	0.091
from	189,451	18	0.51	0.091	we	80,245	43	0.22	0.093
as	181,714	19	0.49	0.093	more	76,388	44	0.21	0.090
be	157,300	20	0.42	0.084	after	75,165	45	0.20	0.091
were	153,913	21	0.41	0.087	us	72,045	46	0.19	0.089
an	152,576	22	0.41	0.090	percent	71,956	47	0.19	0.091
have	149,749	23	0.40	0.092	up	71,082	48	0.19	0.092
his	142,285	24	0.38	0.092	one	70,266	49	0.19	0.092
but	140,880	25	0.38	0.094	people	68,988	50	0.19	0.093

• **Observations**

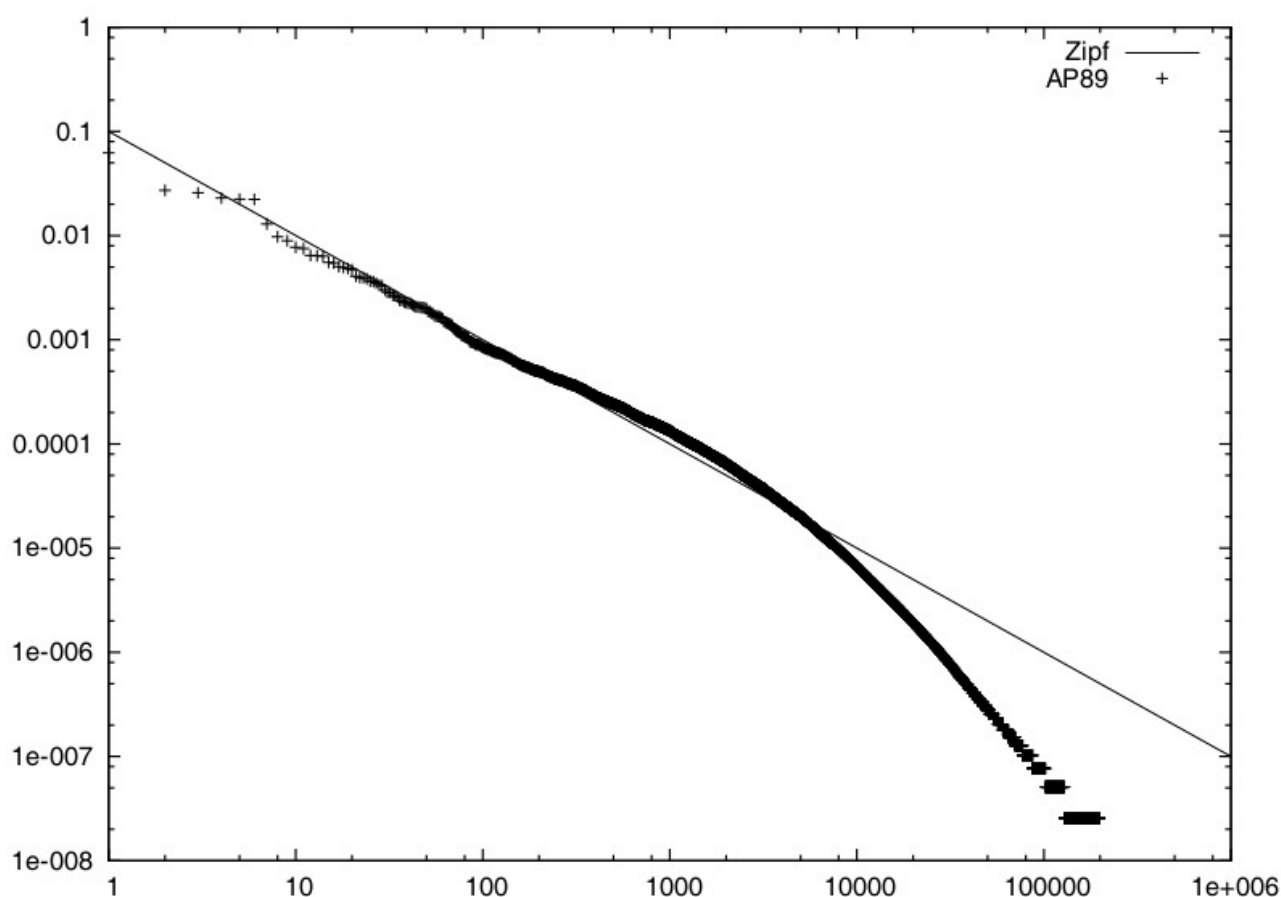
- Zipf 's law is quite accurate, in that the value of $r.P_r$ is approximately constant, and close to 0.1
- The biggest variations are for some of the most frequent words
- In general, Zipf 's law is inaccurate for low and high ranks (high-frequency and low-frequency words) – see some examples for lower frequency words below

<i>Word</i>	<i>Freq.</i>	<i>r</i>	$P_r(\%)$	$r.P_r$
assistant	5,095	1,021	.013	0.13
sewers	100	17,110	.000256	0.04
toothbrush	10	51,555	.000025	0.01
hazmat	1	166,945	.000002	0.04

- log-log plot of the $r.P_r$ values for all words in the AP89 collection given below
- Zipf 's law: A straight line on this graph since

$$\log P_r = \log(c \cdot r^{-1}) = \log c - \log r$$

- Predicted relationship breaks down at high and low ranks (i.e., low and high frequencies)



2.2.2 Vocabulary Growth

- Vocabulary growth is another useful prediction related to word occurrence
- As the size of the corpus grows, new words occur, i.e., vocabulary size also increases
 - However, fewer new words when corpus is already large (as expected from a Zipf distribution for words)
- Relationship above between the size of the corpus and the size of the vocabulary is captured in Heap's Law.

$$v = k \cdot n^{\beta}$$

- Where

v is the vocabulary size (number of unique words)

n is the total number of words in the corpus

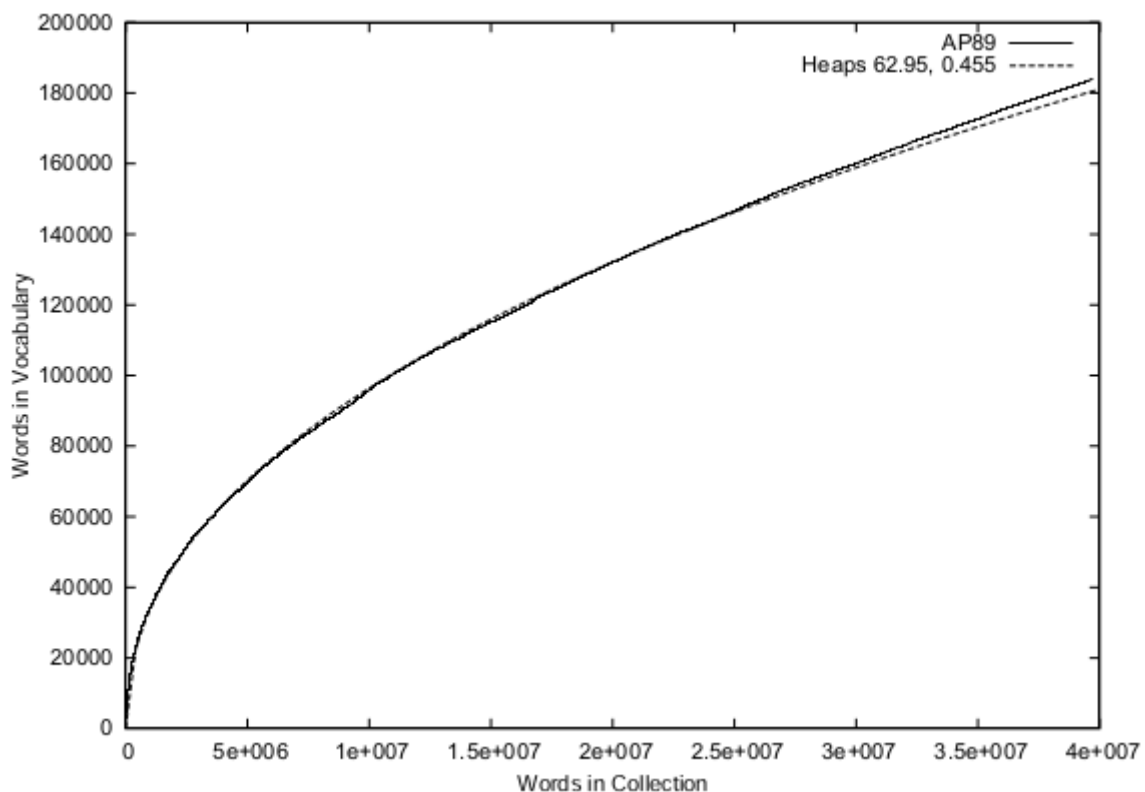
k and β are parameters that vary for each collection (typically, $10 \leq k \leq 100$ and $\beta \approx 0.5$)

- Heaps' law predicts that the number of new words will increase very rapidly when the corpus is small and will continue to increase indefinitely, but at a slower rate for larger corpora

Example

A plot of vocabulary growth for the AP89 collection compared to a graph of Heaps' law with $k = 62.95$ and $\beta = 0.455$.

Clearly, Heaps' law is a good fit. The parameter values are similar for many of the other TREC news collections.



- Example of the accuracy of prediction with Heap's Law
 - If the first 10,879,522 words of the AP89 collection are scanned
 - Heaps' law predicts that the number of unique words will be 100,151
 - The actual value is 100,024
 - Predictions are much less accurate for small numbers of words ($< 1,000$)

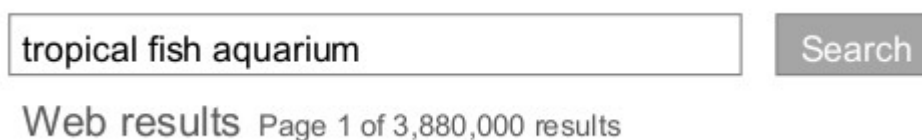
Heaps' Law on the Web

- Heaps' Law works with very large corpora
 - New words occurring even after seeing 30 million!
 - Parameter values different than typical TREC values
- New words come from a variety of sources
 - Spelling errors, invented words (e.g., product, company names), code, other languages, email addresses, people's names, etc.
- Search engines must deal with these large and growing vocabularies

Heaps' Law vs Zipf's Law

- The observed vocabulary growth has a positive correlation with Heaps' law
- Zipf's law, on the other hand, is a poor predictor of high-ranked terms, i.e., Zipf's law is adequate for predicting medium to low ranked terms
- While Heaps' law is a valid model for vocabulary growth of web data, Zipf's law is not strongly correlated with web data

2.2.3 Estimating Collection and Result Set Sizes



The image shows a search bar with the text "tropical fish aquarium" and a "Search" button. Below the search bar, it says "Web results Page 1 of 3,880,000 results".

- Word occurrence statistics can be used to estimate the size of the results from a web search
- In a web search, before the ranked list of results, an estimate of the total number of results is given
 - This is typically a very large number – see interface above

- **Result Set**
 - **Result – Definition:** any document (or web page) that contains **all** of the query words
 - Some search applications will rank documents that do not contain all the query words, but given the huge size of the Web, this is usually not necessary
 - Result set: The ranked list of results

2.2.3.1 *Estimating Result Set Size Based on Words Independence Assumption*

- If we assume that words occur **independently** of each other, then the probability of a document containing all the words in the query is simply the product of the probabilities of the individual words occurring in a document
- For example, if there are three query words a, b, and c, then

$$P(a \cap b \cap c) = P(a) \cdot P(b) \cdot P(c)$$

where $P(a \cap b \cap c)$ is the joint probability, or the probability that all three words occur in a document, and $P(a)$, $P(b)$, and $P(c)$ are the probabilities of each word occurring in a document

- A search engine will always have access to
 - The number of documents that a word occurs in (f_a , f_b , and f_c)
 - These are document occurrence frequencies, not the total number of word occurrences
 - i.e., f_a is the number of documents, containing Word_a, f_b is the number of documents, containing Word_b, etc.
 - The number of documents in the collection (N)
- So the probabilities above can easily be estimated
 - $P(a) = f_a/N$ $P(b) = f_b/N$ $P(c) = f_c/N$
 - Hence,

$$f_{abc} = N \cdot f_a/N \cdot f_b/N \cdot f_c/N = (f_a \cdot f_b \cdot f_c)/N^2$$

f_{abc} is the estimated size of the result set

Example

Document occurrence frequencies for the words “tropical”, “fish”, “aquarium”, and “breeding”, and for combinations of those words in the TREC GOV2 Web collection

<i>Word(s)</i>	<i>Document Frequency</i>	<i>Estimated Frequency</i>
tropical	120,990	
fish	1,131,855	
aquarium	26,480	
breeding	81,885	
tropical fish	18,472	5,433
tropical aquarium	1,921	127
tropical breeding	5,510	393
fish aquarium	9,722	1,189
fish breeding	36,427	3,677
aquarium breeding	1,848	86
tropical fish aquarium	1,529	6
tropical fish breeding	3,629	18

estimates are
off by 1 or 2
orders of
magnitude!

Collection size (N) is 25,205,179

- Estimated sizes of the frequencies of the combinations are based on the independence assumption
- Clearly, this assumption does not lead to good estimates for result size, especially for combinations of three words
 - The problem is that the words in these combinations do not occur independently of each other
 - If we see the word “fish” in a document, for example, then the word “aquarium” is more likely to occur in this document than in one that does not contain “fish”.

2.2.3.2 **Estimating Result Set Size Based on Word Co-occurrence Assumption**

- Better estimates are possible if word **co-occurrence information** is also available from the search engine
 - We would get exact answers for two-word queries (obviously, as we have co-occurrence information)
 - For longer queries, we can improve the estimate by **not assuming independence**

- In general, for three words

$$P(a \cap b \cap c) = P(a \cap b) \cdot P(c|(a \cap b))$$

where

- $P(a \cap b)$ is the probability that the words a and b co-occur in a document
- $P(c|(a \cap b))$ is the conditional probability that the word c occurs in a document given that the words a and b occur in the document
 - If we have co-occurrence information, we can approximate this probability using either $P(c|a)$ or $P(c|b)$, whichever is the larger
- For query “**tropical fish aquarium**” in the table above, this means
 - We estimate the result set size by multiplying the number of documents containing both “tropical” and “aquarium” by the conditional probability that a document contains “fish” given that it contains “aquarium”, i.e.,

$$\begin{aligned} f_{\text{tropical} \cap \text{fish} \cap \text{aquarium}} &= f_{\text{tropical} \cap \text{aquarium}} \cdot f_{\text{fish} \cap \text{aquarium}} / f_{\text{aquarium}} \\ &= 1921 \cdot 9722 / 26480 = 705 \end{aligned}$$

- For the query “tropical fish breeding”

$$\begin{aligned} f_{\text{tropical} \cap \text{fish} \cap \text{breeding}} &= f_{\text{tropical} \cap \text{breeding}} \cdot f_{\text{fish} \cap \text{breeding}} / f_{\text{breeding}} \\ &= 5510 \cdot 36427 / 81885 = 2451 \end{aligned}$$

- These estimates are much better than the ones produced assuming independence, but they are still too low
- Another problem
 - Storing even more information (e.g., the number of occurrences of word triples) in addition to word co-occurrences is very expensive

2.2.3.3 **Estimating Result Set Size Based on Current Result Set**

- It turns out that reasonable estimates of result size can be made using just word frequency and the size of the **current result set**
 - Search engines estimate the result size because they do not rank all the documents that contain the query words (too expensive)
 - Instead, they rank a much smaller subset of the documents that are likely to be the most relevant
 - If we know

- s , The proportion of the total documents that have been ranked
- C , The number of documents found that contain all the query words
- We can simply estimate the result size as C/s
- Underlying assumptions
 - The documents containing all the words are distributed uniformly
 - Document-at-a-time processing, where the inverted lists for all query words are processed at the same time, giving complete document scores
- The proportion of documents processed is measured by the proportion of the documents containing the least frequent word that have been processed, since all results must contain that word

Example

- Query “tropical fish aquarium” used to rank GOV2 documents in the Galago search engine (Galago is a toolkit for experimenting with text search)
 - After processing 3,000 out of the 26,480 documents that contain “aquarium”
 - The number of documents containing all three words is 258
 - This gives a result size estimate of $258 / (3,000 \div 26,480) = 2,277$.
 - After processing just over 20% of the documents, the estimate is 1,778 (compared to the actual figure of 1,529)
- For the query “tropical fish breeding”
 - The estimates after processing 10% and 20% of the documents that contain “breeding” are 4,076 and 3,762 (compared to 3,629)
- These estimates
 - Are quite accurate
 - Do not require knowledge of the total number of documents in the collection

2.2.4 Estimating Collection Size

- Estimating the total number of documents stored in a search engine is of significant interest to both academia (how big is the Web?) and business (which search engine has better coverage of the Web?)

2.2.4.1 Simple method: use independence model

- If a and b are two words that occur independently, then

$$f_{ab}/N = f_a/N \cdot f_b/N$$

$$N = (f_a \cdot f_b) / f_{ab}$$

- But as pointed out earlier
 - The word independence assumption is not valid as words do not generally occur independently of each other
- To get a reasonable estimate of N, we can be more careful about the choice of query words
- Example
 - The following words and word co-occurrences in GOV2
 - “lincoln” (document frequency $f_{\text{lincoln}} = 771,326$)
 - “tropical” (document frequency $f_{\text{tropical}} = 120,990$)
 - “tropical lincoln” (document frequency $f_{\text{lincoln} \cap \text{tropical}} = 3,018$)
 - We would expect the words in the query “tropical lincoln” are less semantically related (and so, more independent) than most other word pairs in the collection
 - So, we can estimate the size of the collection as $N = (120,990 \cdot 771,326) / 3,018 = 30,922,045$
 - This is quite close to the actual number of 25,205,179

2.3 Document Parsing

2.3.1 Overview

- Document parsing involves the recognition of the content and structure of text documents
- Primary content of most documents:
 - Words
 - okenization (or lexical analysis): Recognizing each word occurrence in the sequence of characters in a document
- Other types of content in a document
 - Metadata:
 - Information about a document that is not part of the text content

- Includes
 - Attributes such as date and author
 - The tags that are used by markup languages to identify document components
- The parser uses the tags and other metadata to interpret the document's structure based on the syntax of the markup language (syntactic analysis) and to produce a representation of the document that includes both the structure and content
- Images, graphics, code, and tables

2.3.2 Tokenizing

- Forming words from sequence of characters
- Surprisingly complex in English, can be harder in other languages
- Tokenization strategy of early IR systems
 - Any sequence of alphanumeric characters of length 3 or more
 - Terminated by a space or other special character
 - Upper-case changed to lower-case
- Example (Using the Early IR Approach)
 - The document (string)
 - “Bigcorp's 2007 bi-annual report showed profits rose 10%.”
 - Becomes
 - “bigcorp 2007 annual report showed profits rose”
 - Too simple for search applications or even large-scale experiments
 - Why?
 - To much information lost
 - Small decisions in tokenizing can have major impact on the effectiveness of some queries

Tokenizing Problems

- Small words can be important in some queries, usually in combinations
 - xp, bi, pm, cm, el paso, kg, ben e king, master p, world war II
- Both hyphenated and non-hyphenated forms of many words are common
 - Sometimes hyphen is not needed
 - e-bay, wal-mart, active-x, cd-rom, t-shirts
 - Other times, hyphens should be considered either as part of the word or a word separator
 - winston-salem, mazda rx-7, e-cards, pre-diabetes, t-mobile, spanish-speaking
- Special characters are an important part of tags, URLs, code in documents
- Capitalized words can have different meaning from lower case words

- Bush, Apple, House, Senior, Time, Key
- Apostrophes can be a part of a word/possessive, or just a mistake
 - rosie o'donnell, can't, don't, 80's, 1890's, men's straw hats, master's degree
- Numbers can be important, including decimals
 - Nokia 3250, top 10 courses, united 93, quicktime 6.5 pro, 92.3 the beat
- Periods can occur in numbers, abbreviations, URLs, ends of sentences, and other situations
 - I.B.M., Ph.D., cs.umass.edu, F.E.A.R.
- Note: tokenizing steps for queries must be identical to steps for documents

Tokenizing Process

- First step is to use parser to identify appropriate parts of doc (e.g., markup/tags in markup language) to tokenize
- An approach: defer complex decisions to other components, such as stopping, stemming, & query transformation
 - Word is any sequence of alphanumeric characters, terminated by a space or special character, converted to lower-case
 - Everything indexed
 - Example: 92.3 → 92 3 but search finds document with 92 and 3 adjacent
 - To enhance the effectiveness of query transformation, incorporate some rules into the tokenizer to reduce dependence on other transformation components
- Tokenizing process not that different from simple tokenizing process used in the past
- Examples of rules used with TREC
 - Apostrophes in words ignored
 - o'connor → oconnor bob's → bobs
 - Periods in abbreviations ignored
 - I.B.M. → ibm Ph.D. → phd

2.3.3 Stopping

- Function words (conjunctions, prepositions, articles)
 - Have little meaning on their own
 - High occurrence frequencies
 - Treated as stopwords (i.e., text processing stops when these words are detected & removed hereafter)

- Reduce index space
- Improve response time
- Improve effectiveness
- Can be important in combinations
 - e.g., “to be or not to be”
- Stopword list can be created from high-frequency words or based on a standard list
- Lists are customized for applications, domains, and even parts of documents
 - “click” is a good stopword for anchor text
- One of the policies is to index all words in docs, & make decisions about which words to use at query time

2.3.4 Stemming

2.3.4.1 Overview

- There are many morphological variations of words to convey a single idea
 - Inflectional – i.e., form (plurals, tenses)
 - Derivational (making verbs into nouns, etc.)
- In most cases, these have the same or very similar meanings
- Stemmers attempt to reduce morphological variations of words to a common stem
 - Usually involves removing suffixes
- Can be done at indexing time/as part of query processing (like stopwords)
- Two basic types of stemmer
 - Dictionary-based: uses lists of related words
 - Algorithmic: uses program to determine related words
- Algorithmic stemmers
 - Suffix-s: remove ‘s’ endings assuming plural e.g., cats → cat, lakes → lake, wiis → wii
 - Some false positives
 - Find a relationship when none exists
 - e.g., ups → up
 - Many false negatives
 - Fail to find term relationship
 - e.g., countries → countrie – instead of country

2.3.4.2 Porter Stemmer

- An algorithmic stemmer used in IR experiments since the 70's
- Consists of a series of rules designed to extract the longest possible suffix at each step, e.g.,
 - Step 1a:
 - Replace sses by ss (e.g., stresses → stress)
 - Delete s if the preceding word part contains a vowel not immediately before s (e.g., gaps → gap, gas → gas)
 - Replace ied or ies by i if preceded by > 1 letter; otherwise by ie (e.g., ties → tie, cries → cri)
 - Effective in TREC
 - Produces stems not words
 - Makes a number of errors and difficult to modify

Errors of Porter Stemmer

- It is difficult to capture all the subtleties of a language in a simple algorithm

<i>False positives</i>	<i>False negatives</i>
organization/organ	european/europe
generalization/generic	cylinder/cylindrical
numerical/numerous	matrices/matrix
policy/police	urgency/urgent
university/universe	create/creation
addition/additive	analysis/analyses
negligible/negligent	useful/usefully
execute/executive	noise/noisy
past/paste	decompose/decomposition
ignore/ignorant	sparse/sparsity
special/specialized	resolve/resolution
head/heading	triangle/triangular

- False positives – No Relationship False negatives – Fail to Find a Relationship
- Porter2 stemmer addresses some of these issues
- Approach has been used with other languages

2.3.5 Phrases and N-grams

Read the text

2.4 Link Analysis

Links are a key component of the Web

- Important for navigation, but also for search
 - e.g., `<a href="http://www.example.com">Example Website</a>`
 - “Example website” is the anchor text
 - “http://example.com” is the destination link
 - both are used by search engines

2.4.1 Anchor Text

- Describe the content of the destination page
 - i.e., collection of anchor text in all links pointing to a page used as an additional text field
- Anchor text tends to be short, descriptive, and similar to query text
- Retrieval experiments have shown that anchor text has significant impact on effectiveness for some types of queries
 - even more than PageRank

2.4.2 PageRank

- Billions of web pages, some more informative than others
- Links can be viewed as information about the popularity (authority?) of a web page
 - Can be used by ranking algorithms
- Inlink count could be used as simple measure
- Link analysis algorithms like PageRank provide more reliable ratings, but more susceptible to link spam
- **PageRank of a page is the probability that the “random surfer” will be looking at that page**
 - Links from popular pages increase PageRank of pages they point to, i.e., links tend to point to popular pages

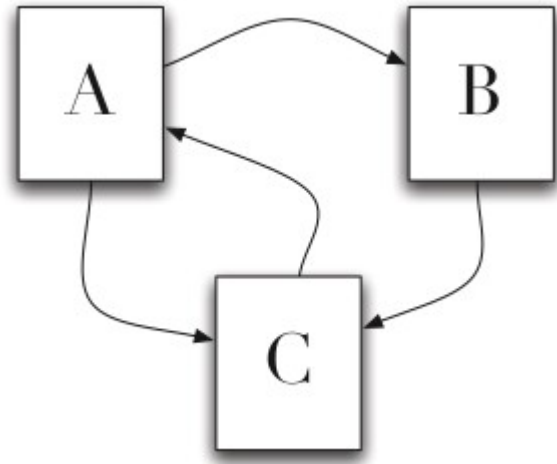
Page Rank for Page c

$$PR(C) = \frac{PR(A)}{2} + \frac{PR(B)}{1}$$

More generally

$$PR(u) = \sum_{v \in B_u} \frac{PR(v)}{L_v}$$

- B_u is the set of pages that point to u
- L_v is the number of outgoing links from page v (not counting duplicate links).

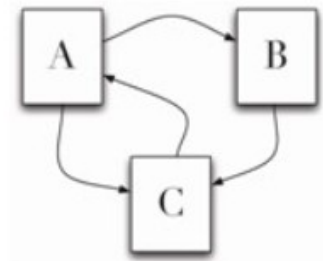


Don't know *PageRank values* at start

Example. Assume equal values of $1/3$, then

- 1st iteration: $PR(C) = 0.33/2 + 0.33/1 = 0.5$
 $PR(A) = 0.33/1 = 0.33$
 $PR(B) = 0.33/2 = 0.17$
- 2nd iteration: $PR(C) = 0.33/2 + 0.17/1 = 0.33$
 $PR(A) = 0.5/1 = 0.5$
 $PR(B) = 0.33/2 = 0.17$
- 3rd iteration: $PR(C) = 0.5/2 + 0.17/1 = 0.42$
 $PR(A) = 0.33/1 = 0.33$
 $PR(B) = 0.5/2 = 0.25$

Converges to $PR(C) = \underline{0.4}$
 $PR(A) = \underline{0.4}$
 $PR(B) = \underline{0.2}$



$$PR(u) = \sum_{v \in B_u} \frac{PR(v)}{L_v}$$